Project Report: YouTube to Audio Converter

Course: CSE1021 Introduction to Problem Solving and Programming

Slot : C21+F11+F12

Faculty Name: M K Jayanthi

School: SCAI

1. Introduction & Motivation

Project Overview This project is a Command-Line Interface (CLI) tool developed in Python that automates the process of downloading YouTube videos and converting them into high-quality audio files. The application accepts a YouTube URL and a desired audio format (like MP3 or FLAC), handles the download via yt-dlp, and processes the conversion using FFmpeg.

Why I Built This The motivation was personal frustration. I frequently need to save educational lectures and specific music tracks for offline listening. The existing online converters are often plagued with intrusive popup ads, slow download queues, and limit the audio quality. I wanted a clean, local solution that runs directly in my terminal, respects my privacy, and gives me control over the file format without any "premium" paywalls.

2. Problem Statement & Requirements

The Problem Users needing offline audio currently face:

- **Ad-heavy interfaces:** Risk of malware from sketchy converter sites.

- **Internet dependency:** You can't convert a file you've already watched without re-uploading or processing it online.
- **Restricted formats:** Most sites only offer low-quality MP3s.

Functional Requirements To solve this, the tool was designed to:

- **Validate Inputs:** robustly check if a link is a valid standard URL, Short, or Mobile link.
- **Auto-Manage Files:** Automatically create a "downloads" directory and sanitize filenames (removing illegal characters) to prevent save errors.
- **Format Flexibility:** Support MP3, WAV, FLAC, AAC, OGG, and M4A.
- **Default Behavior:** If the user is lazy and doesn't specify a format, default to MP3.

Non-Functional Requirements

- **Portability:** The script must be a single file that runs on Windows, macOS, and Linux.
- **Dependencies:** It should rely on robust, maintained libraries (yt-dlp) rather than writing a downloader from scratch, which would be prone to breaking whenever YouTube updates its frontend.

3. System Architecture & Design

The system follows a simple three-layer architecture. I kept it contained in a single Python file to make it easy to share and run.

Layer 1: The CLI (User Interface) This handles argument parsing using Python's sys library. It acts as the gatekeeper, ensuring arguments are present before passing data to the logic layer.

Layer 2: The Business Logic (Converter Class) This is where the work happens. It contains the regex patterns for URL validation and a dictionary mapping user-friendly format names (like "mp3") to the specific codecs required by FFmpeg.

Layer 3: External Processing

- **yt-dlp:** Handles the handshake with YouTube, bypassing the video stream to download only the best available audio track.

- **FFmpeg:** acting as the backend processor to convert the raw audio stream into the user's requested container.

[Insert System Architecture Diagram Here] *(Diagram showing User -> CLI -> Python Script -> yt-dlp -> FFmpeg -> Storage)*

Key Design Decisions

- **Why yt-dlp?** I initially considered pytube, but it is frequently unmaintained and breaks often. yt-dlp is the community standard for reliability.
- **Why CLI?** A GUI (Graphical User Interface) would add unnecessary bloat for a tool that essentially does one job. A CLI is faster to develop and easier to automate in batch scripts later. Also I hate gui for some reason
- **Regex for Validation:** Instead of letting the downloader crash on bad URLs, I implemented a regex check upfront to give immediate, helpful feedback to the user.

4. Implementation Details

Technology Stack

- **Language:** Python 3.x
- **Core Library:** yt-dlp (embedded via Python wrapper)
- **Conversion Engine:** FFmpeg (must be installed on the system path)

Code Structure The code is organized into a Converter class. The critical method is download(), which sets up the configuration options for yt-dlp.

```
# Pseudo-code snippet of the configuration logic
ydl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': os.path.join(self.download_dir, '%(title)s.%(ext)s'),
    'postprocessors': [{
        'key': 'FFmpegExtractAudio',
        'preferredcodec': format,  # User input
    }],
}
```

Data Flow

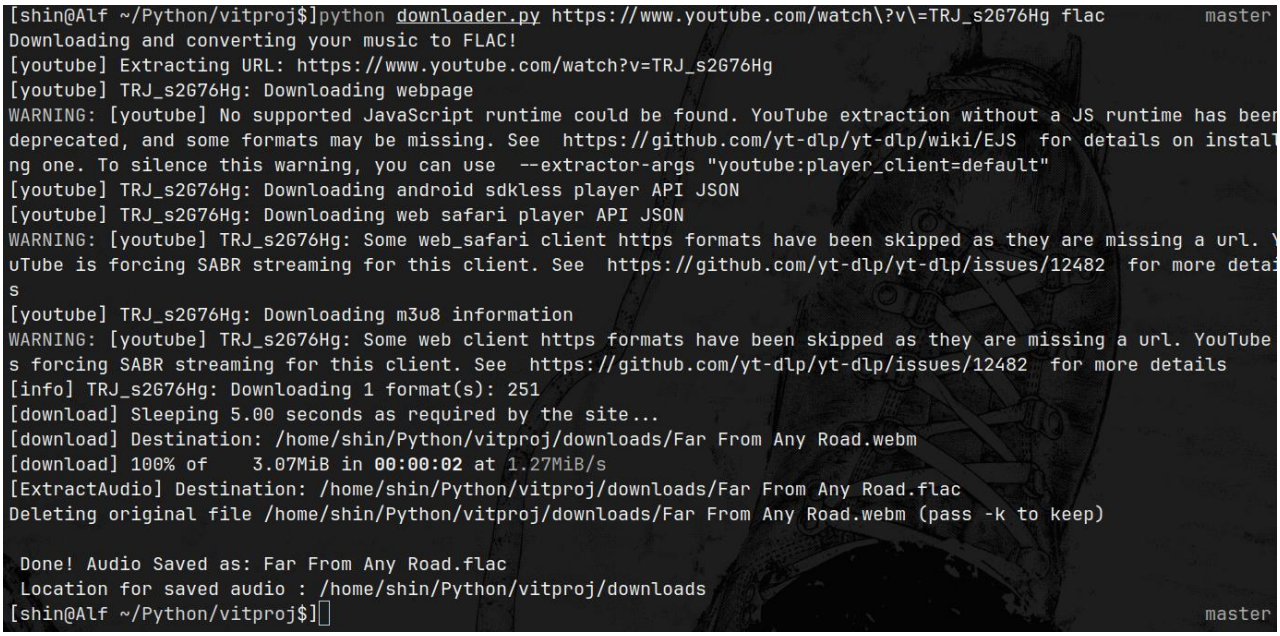
1. User inputs: python main.py [URL] [FORMAT]

2. Script verifies URL against regex pattern ^(https?\:\/\/)?(www\.youtube\.com|youtu\.?be)\/.+$
3. Script checks if the requested format exists in the allowed list.
4. yt-dlp fetches the audio stream.
5. Post-processor triggers FFmpeg to convert the stream.
6. File is moved to the /downloads folder and the temp file is deleted.

5. Testing & Results

I performed manual black-box testing to ensure the tool was robust.

- **Standard Case:** Tested with a standard 3-minute music video. Downloaded and converted to MP3 successfully.
- **Edge Case (Shorts):** Tested with a YouTube Shorts URL. The regex successfully identified it, and the audio was extracted.
- **Edge Case (Invalid Input):** Tried entering "https://www.google.com/search?q=google.com" and random text. The system correctly returned "Invalid YouTube URL".
- **Format Testing:** I downloaded the same video in FLAC (lossless) and MP3 to compare. The file sizes reflected the difference (FLAC was ~30MB, MP3 was ~4MB), confirming the conversion worked.



(Screenshot showing the progress bar and "Done! Audio Saved" message)

6. Challenges & Learnings

Challenge 1: The FFmpeg Dependency The biggest headache was realizing that yt-dlp doesn't include the conversion engine itself. On my first test run on a different machine, the script crashed because FFmpeg wasn't in the system PATH.

- *Solution:* I couldn't "fix" this in code without making the app huge, so I added a specific check in the code to see if FFmpeg is installed and, if not, print a helpful "Please install FFmpeg" message instead of a raw stack trace.

Challenge 2: Filename Handling Some YouTube titles contain characters like |, /, or emojis that Windows filesystems hate.

- *Solution:* I learned that yt-dlp handles most of this, but I had to ensure my output template string was set up correctly to let the library sanitize the filename before writing to the disk.

Challenge 3: URL Variations I underestimated how many ways a YouTube link can look (mobile links, shorts, timestamps). My initial regex was too strict.

- *Learning:* I had to spend time on regex101.com refining the pattern to be permissive enough for valid links but strict enough to block garbage inputs.

7. Future Enhancements

If I were to continue this project, I would focus on:

1. **Playlist Support:** Currently, it only handles one video at a time. Passing a playlist URL should ideally iterate through all videos.
2. **Metadata Tagging:** It would be great if the script automatically grabbed the thumbnail and added it as the album art to the MP3 file, along with the artist name.
3. **Batch Processing:** Allowing the user to point the tool at a text file containing a list of URLs to download overnight.

8. Conclusion

This project was a great exercise in system design. While the code itself isn't massive, the value lies in integrating existing powerful tools (yt-dlp and FFmpeg) into a cohesive, user-friendly package. I successfully created a tool that solves my initial problem: I can now archive audio from YouTube efficiently, privately, and without ads.